



Ohm

An Agentforce agent that audits Agentforce agents.

Ohm reads the agents you have published in your org, finds the inference work that does not need to happen, and shows what you get back when you fix it: energy, carbon and water, with every assumption in the open.

Submission Dreamforce 2026 · Agents for Good Hackathon · Builder Track

Team representative Dominick DeFazio

Built on Agent Script, Prompt Builder (GPT 5.5), Models API, Metadata API, Apex, Lightning Web Components

Status Running live · 287 Apex tests and 218 Jest tests passing

1. The problem

Every Agentforce agent run costs compute, and compute costs electricity, carbon and water. Much of that work is necessary. A meaningful share is not.

Three patterns account for most of the waste we see. A policy pasted repeatedly into a prompt is re-read on every request, forever. A prompt asked to produce eight variations when the user wanted one generates seven throwaways. A prompt asked to sort a list and compute times burns inference on work a few lines of Apex would do exactly, every time, for effectively nothing.

None of this is visible today. Agentforce has no footprint view, no per-agent token report, and no mechanism to tell a builder which agent is wasteful or what to do about it. Waste that is tolerable across ten agents becomes a material line item across ten thousand.

2. What Ohm does

Ohm asks three questions about every published agent, grades each one, and prices the answer.

Category	The question	Typical finding
Input	Is the model reading more than it needs?	Policy text repeated six times in one prompt
Output	Is it writing more than anyone asked for?	Eight drafts and four formats when one plain-text draft is used
Calls	Is it doing work plain code should do?	Arithmetic, sorting and overlap checks inside a prompt

Each bundle receives a grade of **A** (clean in the reviewed evidence), **B** (a change worth making), or **C** (an explicit budget exceeded). Where evidence is missing, the category returns **Unrated** and stays visible rather than being scored.

Alongside the grades, Ohm models the bundle's annual environmental footprint and, more usefully, what applying its recommendations would avoid. Every recommendation carries three things a builder actually needs: what to change, what must stay intact, and how to validate it. One click turns any recommendation into a standard Salesforce Task.

3. How it works

Six stages, all inside your org. The gate in the middle is the design.

Discover Published Agent Script bundles	Retrieve Exact published source via the	Review A Prompt Builder template on GPT	Verify Apex checks every quote	Recommend Change, keep, validate –	Act A real Salesforce Task, keyed to the
--	--	--	---	---	---

and the prompt templates their actions call.

Metadata API, using Ohm's own credential. Every artifact pinned by content hash.

5.5 reads the source and returns findings that quote it.

against the pinned source, confirms freshness, applies fixed rating rules, saves once.

generated from the saved review, never from a fresh model call.

review so it cannot duplicate.

The model does not get the last word. If any quoted passage fails to match the retrieved source, or the source changed mid-run, the entire review is discarded and nothing is written. A partial or unverifiable review is never saved. This fired in production testing and the retry succeeded, which is precisely the intended behaviour.

Why verification rather than trust

An LLM grading someone's work is a high-trust operation. A single invented quote or hallucinated finding costs a builder real time and costs the tool its credibility permanently. So the model supplies interpretation and evidence, and deterministic Apex supplies the verdict. The model cannot invent a grade, cannot cite source that does not exist, and cannot cause a partial write.

4. Architecture

Experience – Lightning Web Components

Overview, Bundles, Recommendations, How it works, Ask Ohm, and the environmental footprint strip.

Services – Apex

Discovery, source retrieval, review and verification, recommendations and Tasks, the footprint engine, and the assistant. Five invocable actions expose these to the Ohm agent.

Platform – Salesforce and Agentforce

Agent Script authoring bundles, Prompt Builder on GPT 5.5, the Models API, the Metadata API, an External Client App with client-credentials flow behind a Named Credential, custom objects and standard Tasks.

Ohm as an agent

Ohm is itself a published Agentforce agent with five custom Apex actions, so the whole product is reachable in conversation: `list_bundles`, `get_recommendations`, `start_audit`, `audit_status` and `create_task`. Ask which agents waste the most, ask what to change, tell it to audit one now, check the run, and have it create the task.

Source integrity

Retrieval runs through a dedicated External Client App using the client-credentials flow with a fixed integration user, behind a Named Credential. No CLI session or user token is ever stored in the app. Every retrieved

artifact is hashed at retrieval, and that hash is what verification checks against, so a review can never drift from the source it describes.

5. The environmental model

Tokens are measured. Energy is modeled. Ohm never conflates the two.

Nothing in the Salesforce platform, or from the model providers, reports energy consumption per request. Ohm therefore derives energy from source tokens using published figures, and labels every resulting number **Modeled, not measured** with each constant one click away in the interface.

Factor	Central	Band	Source
Energy per 500-token prompt	0.27 Wh	0.24 - 0.30	Epoch AI 2025 median, GPT-4o class
Grid intensity	0.30 g CO2e / Wh	0.125 - 0.475	Published grid averages
Water	1.08 mL / Wh	0.8 - 1.4	Published data-centre figures
Token approximation	chars ÷ 4	–	Standard approximation

Confidence is explicitly labelled Low. Figures scale with a request-volume selector so any claim can be stated with the volume it assumes rather than asserted flat.

6. Results

Four controlled Agent Script bundles, audited live through the source connection and the GPT 5.5 reviewer.

Bundle	Input	Output	Calls	Modeled avoided / year
Customer Support	C	A	A	19.2 kWh (51%)
Sales Follow-up	A	B	A	4.43 kWh (19%)
Schedule Planning	A	A	B	6.09 kWh (25%)
Efficient Schedule Desk (control)	A	A	A	none recommended

30% of fleet footprint avoidable	63%	0 unverified ratings ever saved
-------------------------------------	-----	------------------------------------

	measured input-token reduction, largest fix	
--	--	--

The control

The fourth bundle does the same job as Schedule Planning, built correctly: Apex performs the logic and the prompt only presents the result. It was put in front of the reviewer with no indication that it was a control. It earned straight A grades and produced no recommendations. A reviewer that cannot say "this is fine" is worthless, and this is how we demonstrate ours can.

Behavioural evidence

Using Salesforce's own agent evaluation framework, seven of eight cases passed. The single failure is the most useful result in the project: the prompt performing scheduling arithmetic retained an activity it should have excluded, while the Apex-backed version was correct. Moving that prompt to a stronger model made the case pass. Prompt arithmetic is model-dependent and quietly fragile; code is not. That is the entire argument for the Calls category, demonstrated rather than asserted.

7. Efficiency of Ohm itself

- **One model call per audit.** The full review of a bundle and its linked prompts is a single generation. No chained calls, no agentic loop.
- **Explaining a saved review costs nothing.** Answers are assembled deterministically from the stored review. The most frequent interaction in the product makes no model call.
- **Listing, ranking and volume scaling are pure Apex.** Moving the selector from 300 to 100,000 requests a day is arithmetic on saved data.
- **Discovery and retrieval never call a model.** Metadata API and SOQL only.
- **Failure is terminal, not retried.** An audit that cannot verify its evidence stops rather than burning a second generation.
- **Nothing runs on a schedule.** Audits happen when a person asks for one.

8. Safety boundaries

Ohm has no write path to your agents. It cannot edit source, publish, or deploy. The only record it writes is a Task. A person makes the change, tests it, and publishes it. The draft-rewrite feature is deliberately a proposal that must be copied out by hand, and the interface states that applying it is a separate, human, deliberate act.